EXERCÍCIO - CADASTRO DE PRODUTOS COM SQLITE E LOG

Objetivo geral: praticar persistência de dados usando SQLite (JDBC), tratamento explícito de exceções de acesso ao banco e registro de sucessos/erros em arquivo de log (loja.log).

1. OBJETIVOS DO EXERCÍCIO

- Implementar um cadastro simples de produtos com persistência em SQLite (arquivo loja.db).
- Aplicar tratamento de exceções (try/catch) em todas as interações com o banco de dados.
- Registrar sucessos e erros em um arquivo de log (loja.log) usando a classe FileTools.
- Usar PreparedStatement para comandos SQL parametrizados (segurança e clareza).
- Entregar uma aplicação de linha de comando com um menu básico de operações.

2. O QUE DEVE SER FEITO (PASSO A PASSO)

1. Baixar/adicionar ao projeto as classes fornecidas: FileTools.java e GerenciarBD.java.
2. Garantir o driver JDBC do SQLite no classpath (ex.: **org.xerial:sqlite-jdbc**).
3. Criar a classe Main (ou similar) com um menu:

```
1 - Adicionar produto
2 - Listar todos os produtos
3 - Listar produtos com estoque baixo (≤ limite informado)
4 - Atualizar estoque de um produto (por id)
0 - Sair
```

4. No início da execução, chamar GerenciarBD.criarTabelaSeNaoExistir() e registrar SUCESSO/ERRO em loja.log.
5. Para cada operação do menu: validar entradas, chamar os métodos de GerenciarBD dentro de try/catch, e registrar SUCESSO/ERRO no log via FileTools.
6. Formatar preço com 2 casas decimais na saída.
7. Fechar recursos com try-with-resources (JDBC e I/O).
8. Testar todos os fluxos: inserir, listar, filtrar por estoque, atualizar. Conferir o arquivo loja.log.

3. EXEMPLOS MÍNIMOS DE USO NO MAIN

3.1 CRIAR/VALIDAR TABELA E LOGAR RESULTADO

```
// Dentro do método main

GerenciarBD dao = new GerenciarBD("loja.db");

try {
    dao.criarTabelaSeNaoExistir();
    FileTools.appendLog("loja.log", "SUCESSO", "Tabela verificada/criada.");
} catch (java.sql.SQLException ex) {
    FileTools.appendLog("loja.log", "ERRO", "Falha ao criar/verificar tabela.",
ex);
    System.err.println("Não foi possível preparar a tabela. Encerrando.");
    return;
}
```

3.2 INSERIR PRODUTO COM PREPAREDSTATEMENT (RETORNANDO ID)

```
try (java.sql.Connection c =
java.sql.DriverManager.getConnection("jdbc:sqlite:loja.db")) {
    String sql = "INSERT INTO produtos (nome, preco, quantidade) VALUES (?, ?, ?)";
    try (java.sql.PreparedStatement ps =
        c.prepareStatement(sql,java.sql.Statement.RETURN_GENERATED_KEYS)) {
        ps.setString(1, "Caderno");
        ps.setDouble(2, 15.90);
        ps.setInt(3, 50);
        int linhas = ps.executeUpdate();
        if (linhas > 0) {
            try (java.sql.ResultSet rs = ps.getGeneratedKeys()) {
                if (rs.next()) {
                    long idGerado = rs.getLong(1);
                    FileTools.appendLog("loja.log", "SUCESSO", "Produto inserido
                    (id=" +  idGerado + ").");
                }
            }
        }
    }
} catch (java.sql.SQLException ex) {
    FileTools.appendLog("loja.log", "ERRO", "Falha ao inserir produto.", ex);
}
```

4. APÊNDICES

APÊNDICE A — TRABALHANDO COM ARQUIVOS EM JAVA (PASSO A PASSO)

Passos práticos:

- Escolher a classe/abordagem: Leitura (FileReader+BufferedReader ou Files.readAllLines), Escrita (FileWriter+BufferedWriter, PrintWriter ou Files.write), Gerenciamento (File/Files).
- Abrir o arquivo (leitura ou escrita) — preferir try-with-resources.
- Ler/Escrever o conteúdo (readLine, write/newLine, etc.).
- Fechar (automático no try-with-resources).
- Tratar IOException com try/catch.
- Operações extras: criar/verificar/deletar arquivo com File/Files.

TABELA COMPARATIVA — ARQUIVOS

Objetivo	Classe/Abordagem	Exemplo curto
Criar/Verificar arquivo	File ou Files	new java.io.File("arq.txt").createNewFile();
Escrever (sobrescrever)	FileWriter + BufferedWriter	try (var bw = new java.io.BufferedWriter(new java.io.FileWriter("x.txt"))) { bw.write("..."); }
Escrever (append)	FileWriter (append = true)	new java.io.FileWriter("x.txt", true);
Escrever formatado	PrintWriter	try (var pw = new java.io.PrintWriter("x.txt")) { pw.println("linha"); }
Ler linha a linha	FileReader + BufferedReader	try (var br = new java.io.BufferedReader(new java.io.FileReader("x.txt"))) { String l; while((l=br.readLine())!=null){ } }
Ler tudo (lista de linhas)	Files.readAllLines	var ls = java.nio.file.Files.readAllLines(java.nio.file.Paths.get("x.txt"));
Escrever tudo de uma vez	Files.write	java.nio.file.Files.write(java.nio.file.Paths.get("x.txt"), java.util.Arrays.asList("a","b"));
Deletar arquivo	File.delete()	new java.io.File("x.txt").delete();

APÊNDICE B — PERSISTÊNCIA COM SQLITE EM JAVA (PASSO A PASSO)

- Adicionar o driver JDBC do SQLite ao projeto (ex.: org.xerial:sqlite-jdbc).
- Definir/confirmar o esquema: tabela produtos com (id, nome, preco, quantidade).
- Abrir conexão: URL JDBC "jdbc:sqlite:loja.db" (use try-with-resources).
- Usar PreparedStatement sempre (parametrização com ?).
- INSERT/UPDATE/DELETE → executeUpdate(); SELECT → executeQuery(); casos genéricos → execute().
- Para INSERT com retorno de id: usar Statement.RETURN_GENERATED_KEYS + getGeneratedKeys().
- Tratar SQLException em todas as operações; logar SUCESSO/ERRO via FileTools.
- Transações quando houver várias operações dependentes: setAutoCommit(false), commit(), rollback().
- Mapear ResultSet em DTO (ProdutoDTO) para transportar dados.
- Validar entradas (nome não vazio, preco >= 0, quantidade >= 0).
- Fechar recursos automaticamente com try-with-resources.

APÊNDICE C — O QUE É UM PREPAREDSTATEMENT?

É uma classe da API JDBC que permite executar comandos SQL com parâmetros (placeholders '?'), separando o comando SQL dos dados, tornando o código mais seguro (previne SQL Injection), legível e potencialmente mais eficiente.

Exemplo de uso com INSERT e retorno do id:

```
String sql = "INSERT INTO produtos (nome, preco, quantidade) VALUES (?, ?, ?)";
try (java.sql.Connection c = java.sql.DriverManager.getConnection("jdbc:sqlite:loja.db");
     java.sql.PreparedStatement ps = c.prepareStatement(sql, java.sql.Statement.RETURN_GENERATED_KEYS)) {
    ps.setString(1, "Caderno");
    ps.setDouble(2, 15.90);
    ps.setInt(3, 50);
    int linhas = ps.executeUpdate();
    if (linhas > 0) {
        try (java.sql.ResultSet rs = ps.getGeneratedKeys()) {
            if (rs.next()) {
                long idGerado = rs.getLong(1);
                // usar idGerado...
            }
        }
    }
} catch (java.sql.SQLException ex) {
    // tratar e logar
}
```

APÊNDICE D — PRINCIPAIS MÉTODOS DE EXECUÇÃO EM PREPAREDSTATEMENT

Operação SQL	Método	Retorno
INSERT / UPDATE / DELETE / CREATE TABLE ...	ps.executeUpdate()	int (linhas afetadas)
SELECT	ps.executeQuery()	ResultSet (dados consultados)
Desconhecida/Genérica (raro em CRUD simples)	ps.execute()	boolean (true=ResultSet, false=update)

5. CLASSES DE APOIO FORNECIDAS

5.1 FILETOOLS.JAVA

```
// FileTools.java
// Classe utilitária com métodos estáticos para criar, escrever, ler e deletar
arquivos .txt/.log.
// Todos os métodos são comentados linha a linha para fins didáticos.

import java.io.BufferedWriter;            // Escrita eficiente em arquivos
import java.io.File;                      // Representa caminhos/arquivos
import java.io.FileWriter;                // Escreve caracteres em um arquivo
import java.io.IOException;               // Exceções de I/O
import java.nio.charset.StandardCharsets; // Charset padrão (UTF-8)
import java.nio.file.Files;               // Operações utilitárias com arquivos
import java.nio.file.Path;                // Representa um caminho (NIO)
import java.nio.file.Paths;               // Fábrica de Path
import java.time.LocalDateTime;           // Data/hora atual
import java.time.format.DateTimeFormatter;// Formatação de data/hora
import java.util.List;                    // Lista de linhas

public class FileTools {

    // Garante que o diretório pai exista antes de criar/usar um arquivo.
    private static void ensureParentDirectory(Path path) throws IOException {
        Path parent = path.getParent(); // Diretório pai (pode ser null)
        if (parent != null && !Files.exists(parent)) {
            Files.createDirectories(parent); // Cria diretórios necessários
        }
    }

    // Cria o arquivo se ele não existir.
    public static void createFileIfNotExists(String filePath) throws IOException {
        Path path = Paths.get(filePath);     // Converte String para Path
        ensureParentDirectory(path);         // Garante diretório pai
        if (!Files.exists(path)) {           // Se não existe...
            Files.createFile(path);          // Cria o arquivo vazio
        }
    }

    // Escreve UMA linha no arquivo. append=true adiciona ao final; false
sobrescreve.
    public static void writeLine(String filePath, String line, boolean append)
throws IOException {
```

```
        createFileIfNotExists(filePath);     // Garante existência do arquivo
        try (FileWriter fw = new FileWriter(new File(filePath), append);
             BufferedWriter bw = new BufferedWriter(fw)) {
            bw.write(line);                  // Escreve a linha
            bw.newLine();                    // Quebra de linha do SO
            // try-with-resources fecha bw e fw automaticamente
        }
    }

    // Lê todas as linhas de um arquivo texto e retorna como lista.
    public static List<String> readAll(String filePath) throws IOException {
        Path path = Paths.get(filePath);     // Caminho do arquivo
        createFileIfNotExists(filePath);     // Cria vazio se não existir
        return Files.readAllLines(path, StandardCharsets.UTF_8); // Lê todas as
linhas
    }

    // Deleta o arquivo se existir. Retorna true se deletou; false se não havia
arquivo.
    public static boolean deleteFile(String filePath) throws IOException {
        Path path = Paths.get(filePath);     // Caminho para Path
        return Files.deleteIfExists(path);   // Deleta se existir
    }

    // Retorna timestamp legível no padrão "yyyy-MM-dd HH:mm:ss"
    private static String nowString() {
        LocalDateTime now = LocalDateTime.now();                 // Agora
        DateTimeFormatter fmt = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
        return now.format(fmt);                                  // Formata
    }

    // Escreve linha de log no formato: [dataHora] [NIVEL] mensagem
    public static void appendLog(String logPath, String level, String message) {
        try {
            String line = "[" + nowString() + "] [" + level + "] " + message; //
Monta a linha
            writeLine(logPath, line, true);                                    //
Acrescenta ao final
        } catch (IOException e) {
            System.err.println("Falha ao escrever no log: " + e.getMessage()); //
Fallback no console
        }
    }
```

```
    // Versão de log que inclui classe e mensagem de exceção + primeiro frame da
stack.
    public static void appendLog(String logPath, String level, String message,
Exception ex) {
        String exMsg = (ex != null && ex.getMessage() != null) ? ex.getMessage() :
"sem mensagem";
        String where = (ex != null && ex.getStackTrace().length > 0)
                ? ex.getStackTrace()[0].toString()
                : "sem stack";
        String full = message + " | excecao=" + (ex != null ?
ex.getClass().getSimpleName() : "null")
                + ": " + exMsg + " @ " + where;
        appendLog(logPath, level, full); // Reaproveita método acima
    }
}
```

4.2 GERENCIARBD.JAVA (DAO PARA SQLITE)

```
// GerenciarBD.java
// Classe de acesso a dados (DAO simples) para SQLite.
// Fornece métodos para criar tabela, inserir, buscar e atualizar produtos.
// Todos os métodos são comentados linha a linha para fins didáticos.

import java.sql.Connection;          // Conexão com o banco
import java.sql.DriverManager;       // Cria conexões JDBC
import java.sql.PreparedStatement;   // Comandos SQL parametrizados
import java.sql.ResultSet;           // Resultado de SELECT
import java.sql.SQLException;        // Exceções de banco
import java.sql.Statement;           // Constantes e operações gerais
import java.util.ArrayList;          // Lista concreta
import java.util.List;               // Interface de lista

public class GerenciarBD {

    // URL JDBC do SQLite. Ex.: "jdbc:sqlite:loja.db"
    private final String dbUrl;

    // Construtor recebe caminho do arquivo do banco (ex.: "loja.db")
    public GerenciarBD(String dbFilePath) {
        this.dbUrl = "jdbc:sqlite:" + dbFilePath; // Monta URL JDBC
    }

    // Abre e retorna uma conexão com o banco de dados.
    private Connection getConnection() throws SQLException {
            return DriverManager.getConnection(dbUrl); // Abre conexão via
     DriverManager
    }

    // Cria a tabela 'produtos' se não existir.
    public void criarTabelaSeNaoExistir() throws SQLException {
        final String sql = "CREATE TABLE IF NOT EXISTS produtos (" +
                           "id INTEGER PRIMARY KEY AUTOINCREMENT," +
                           "nome TEXT NOT NULL," +
                           "preco REAL NOT NULL," +
                           "quantidade INTEGER NOT NULL DEFAULT 0" +
                           ")";
        try (Connection conn = getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
```

```
            ps.execute(); // Executa DDL
        }
    }

    // Insere um produto e retorna o id gerado.
    public long inserirProduto(String nome, double preco, int quantidade) throws
SQLException {
        final String sql = "INSERT INTO produtos (nome, preco, quantidade) VALUES
(?, ?, ?)";
        try (Connection conn = getConnection();
             PreparedStatement ps = conn.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, nome);      // Param 1: nome
            ps.setDouble(2, preco);     // Param 2: preco
            ps.setInt(3, quantidade);   // Param 3: quantidade
            ps.executeUpdate();         // Executa INSERT
            try (ResultSet rs = ps.getGeneratedKeys()) { // Recupera chave gerada
                if (rs.next()) {
                    return rs.getLong(1); // Primeiro valor é o id
                }
            }
        }
        return -1L; // Caso raro: não retornou id
    }

    // Retorna todos os produtos.
    public List<ProdutoDTO> listarTodos() throws SQLException {
        final String sql = "SELECT id, nome, preco, quantidade FROM produtos ORDER
BY id";
        List<ProdutoDTO> lista = new ArrayList<>();
        try (Connection conn = getConnection();
             PreparedStatement ps = conn.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) { // Percorre linhas
                ProdutoDTO p = new ProdutoDTO(
                        rs.getInt("id"),
                        rs.getString("nome"),
                        rs.getDouble("preco"),
                        rs.getInt("quantidade")
                );
                lista.add(p);
            }
        }
```

```
        return lista;
    }

    // Retorna produtos com estoque <= limite informado.
    public List<ProdutoDTO> listarPorEstoqueAte(int limite) throws SQLException {
        final String sql = "SELECT id, nome, preco, quantidade FROM produtos WHERE
quantidade <= ? ORDER BY quantidade, id";
        List<ProdutoDTO> lista = new ArrayList<>();
        try (Connection conn = getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, limite);             // Define limite
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    ProdutoDTO p = new ProdutoDTO(
                            rs.getInt("id"),
                            rs.getString("nome"),
                            rs.getDouble("preco"),
                            rs.getInt("quantidade")
                    );
                    lista.add(p);
                }
            }
        }
        return lista;
    }

    // Atualiza a quantidade de um produto pelo id. Retorna linhas afetadas (0 ou
1).
    public int atualizarQuantidade(int id, int novaQuantidade) throws SQLException
{
        final String sql = "UPDATE produtos SET quantidade = ? WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, novaQuantidade); // Nova quantidade
            ps.setInt(2, id);             // ID alvo
            return ps.executeUpdate();    // Executa UPDATE
        }
    }

    // (Opcional) Atualiza o preço de um produto pelo id.
    public int atualizarPreco(int id, double novoPreco) throws SQLException {
        final String sql = "UPDATE produtos SET preco = ? WHERE id = ?";
        try (Connection conn = getConnection();
```

```
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setDouble(1, novoPreco);   // Novo preço
            ps.setInt(2, id);             // ID alvo
            return ps.executeUpdate();    // Executa UPDATE
        }
    }

    // DTO simples para transportar dados.
    public static class ProdutoDTO {
        public final int id;
        public final String nome;
        public final double preco;
        public final int quantidade;

        public ProdutoDTO(int id, String nome, double preco, int quantidade) {
            this.id = id;             // Campo id
            this.nome = nome;         // Campo nome
            this.preco = preco;       // Campo preco
            this.quantidade = quantidade; // Campo quantidade
        }

        @Override
        public String toString() {
            return id + ";" + nome + ";" + preco + ";" + quantidade;
        }
    }
}
```